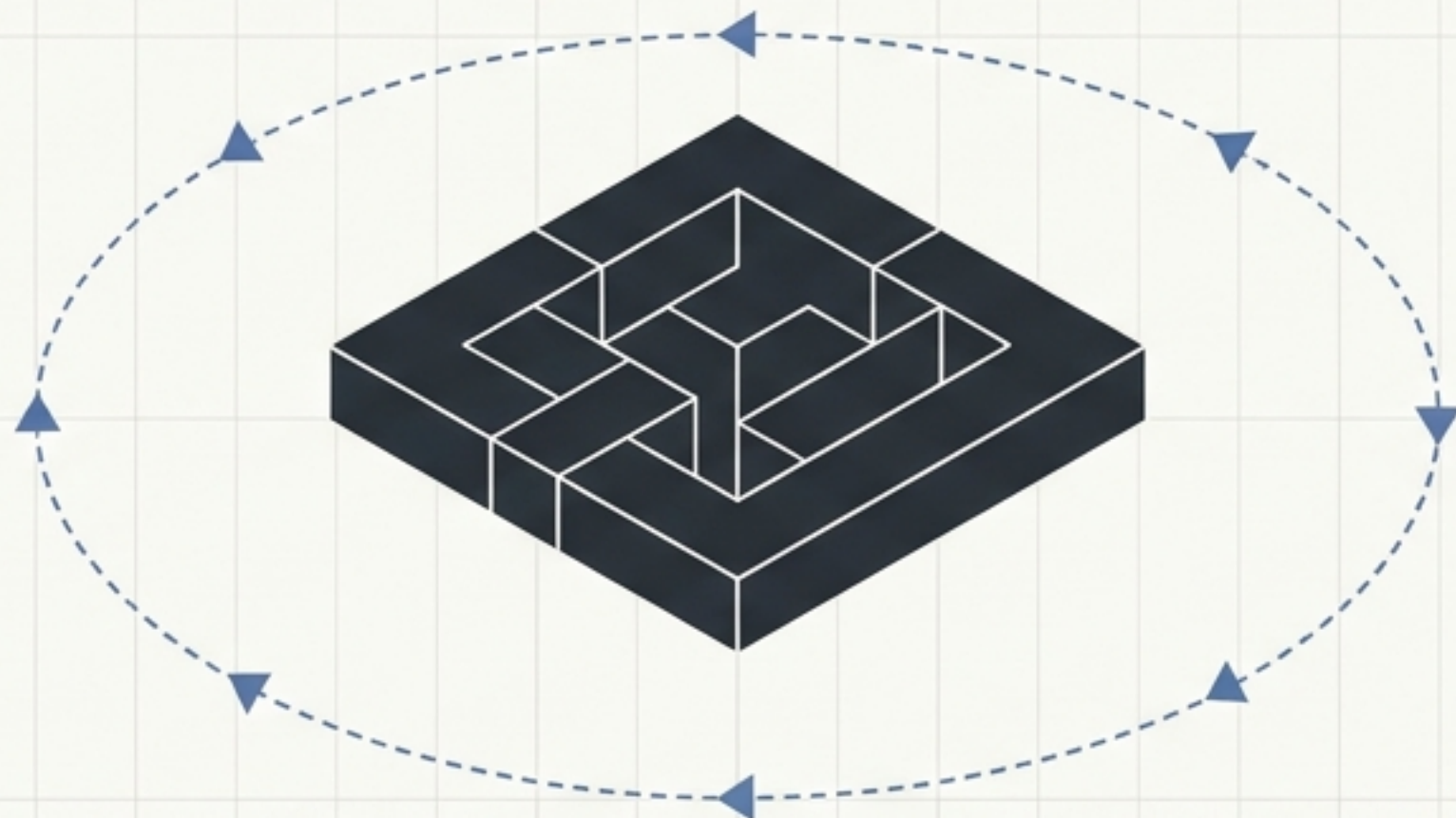


构建无人值守的 AI 软件工厂

基于 Claude Code /loop 与 Comet 框架的自动化架构设计与实战指南



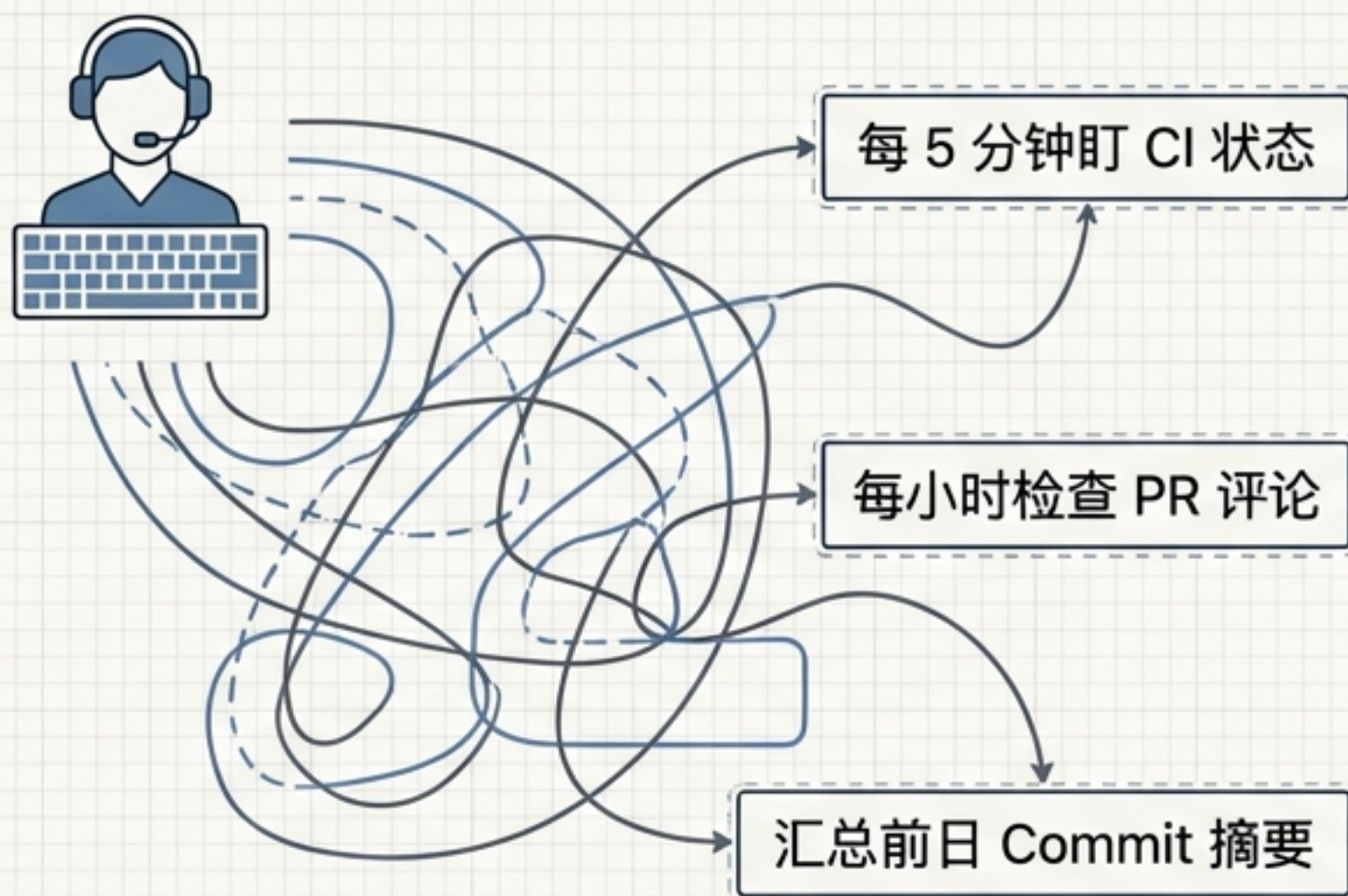
VERSION: 2026-06

TARGET: SESSION_SCOPED_CRON

STATUS: UNATTENDED_MODE_ENABLED

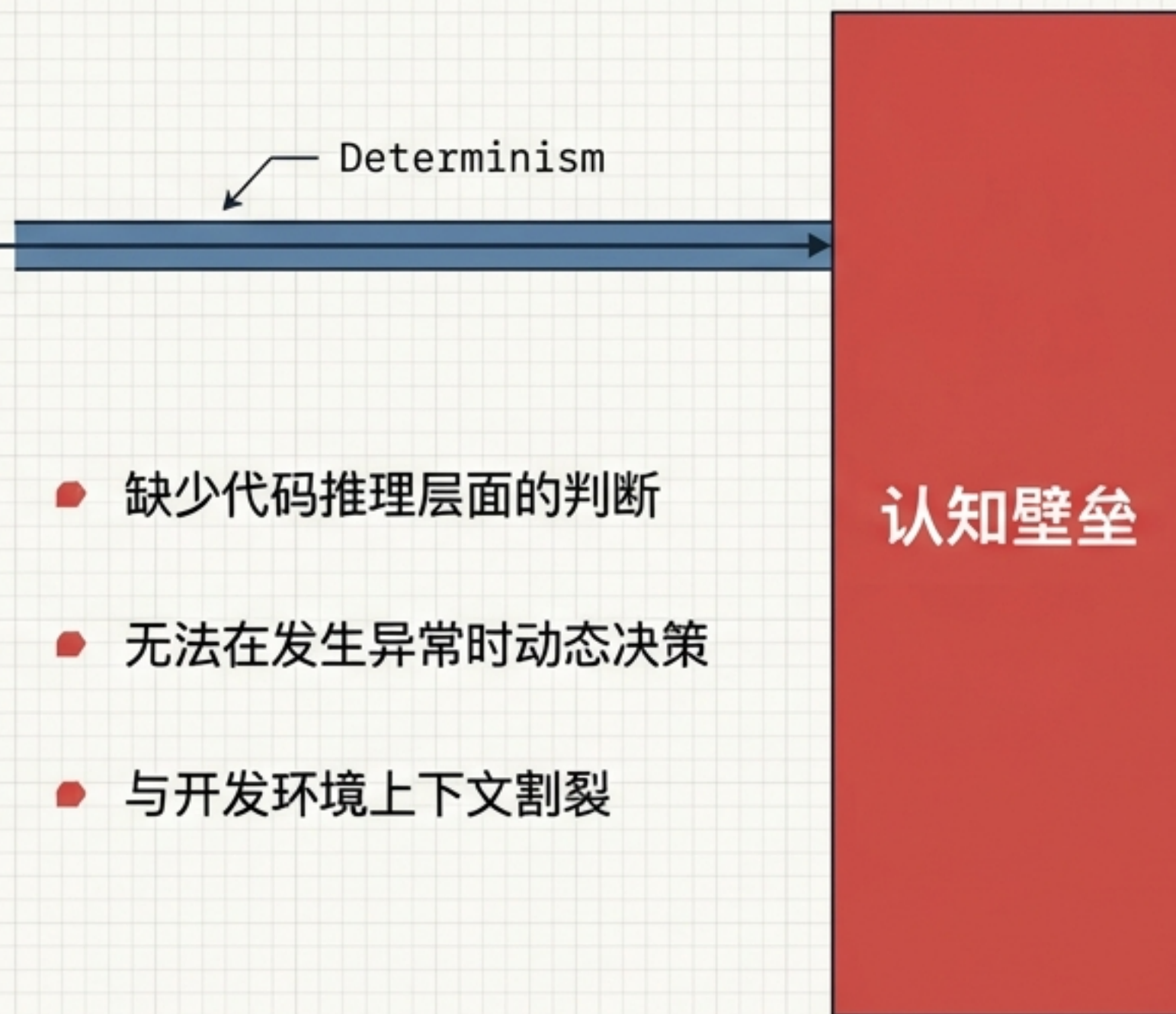
重复劳动的痛点与传统工具的认知壁垒

The Manual Burden



结论：确定性、周期性、但耗费人类精力的任务。

The Deterministic Wall



自动化演进：为什么我们需要 /loop?

方案	智能决策能力	上下文感知	部署摩擦力	核心定位
System Cron	✗ 无	✗ 丢失环境	⚠ 需配认证与变量	系统级定时脚本
Shell 循环	✗ 无	✓ 当前终端	✓ 低	简单确定性轮询
CI 定时触发	✗ 依赖脚本	✗ 独立沙箱	⚠ 需改 CI 配置	仅限 CI 环境任务
/loop	✓ 完整 AI 推理	✓ 继承当前会话	✓ 零配置内置	有人值守的 长期任务推进

解析：/loop 引擎的底层架构

Engine Core

会话级定时器 (Session-Scoped Cron)

保持完整对话上下文。

进程绑定 (Lifecycle Bound)

随 Claude Code 进程启动与退出。

Session

/loop

并发限制 (Capacity)

单会话最多共存 50 个独立定时任务。

闭环迭代 (Iteration)

单次循环 = Prompt -> 工具调用 -> 响应反馈。

生命周期 (TTL)

7 天后自动终止过期。

运行时态：三种自动巡航模式

模式 A：固定间隔 (Fixed)

`/loop 5m check deploy`

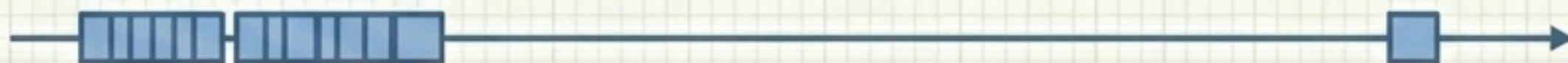
将间隔转换为精确 Cron，适合持续轮询。



模式 B：动态自选 (Dynamic)

`/loop check PR`

AI 根据事件活跃度自选等待时间。活跃时高频，无事时休眠。



模式 C：内置维护
(Maintenance)

`/loop (无参数)`

执行内置链条：处理未完工作 → 照看当前 PR → 清理排查 Bug。



注入自定义逻辑: `.claude/loop.md`

[🏆 Highest Priority]
`.claude/loop.md` (项目级作用域)

支持 热重载 (Hot Reload) ——
保存修改, 下次迭代立即生效。
大小限制 < 25KB。

[🥇 Fallback]
~/.claude/loop.md
(用户全局作用域)

loop.md

自定义 /loop 行为

- 检查所有未提交的 Git 更改。
- 运行 `npm run test` 确保没有破坏性修改。
- 汇总结果并提醒我。

纯 Markdown 定义, 无固定语法。
自由编写测试规则、Git 规范或清理流程。

权限拦截矩阵：实现真正的无人值守

⚠️ 默认模式下 /loop 会被 Allow this command? 弹窗彻底卡死。

权限拦截矩阵			
模式 (Mode)	文件编辑	Bash 命令	适用性
default	❌ 弹窗拦截	❌ 弹窗拦截	❌ 彻底卡死
acceptEdits	✅ 自动放行	❌ 弹窗拦截	⚠️ 仅限写文件
auto	✅ AI 分类器	✅ AI 分类器	👑 最佳选择
dontAsk	❌ 白名单外拒绝	❌ 白名单外拒绝	⚠️ 仅 CI 场景

auto mode AI 分类器

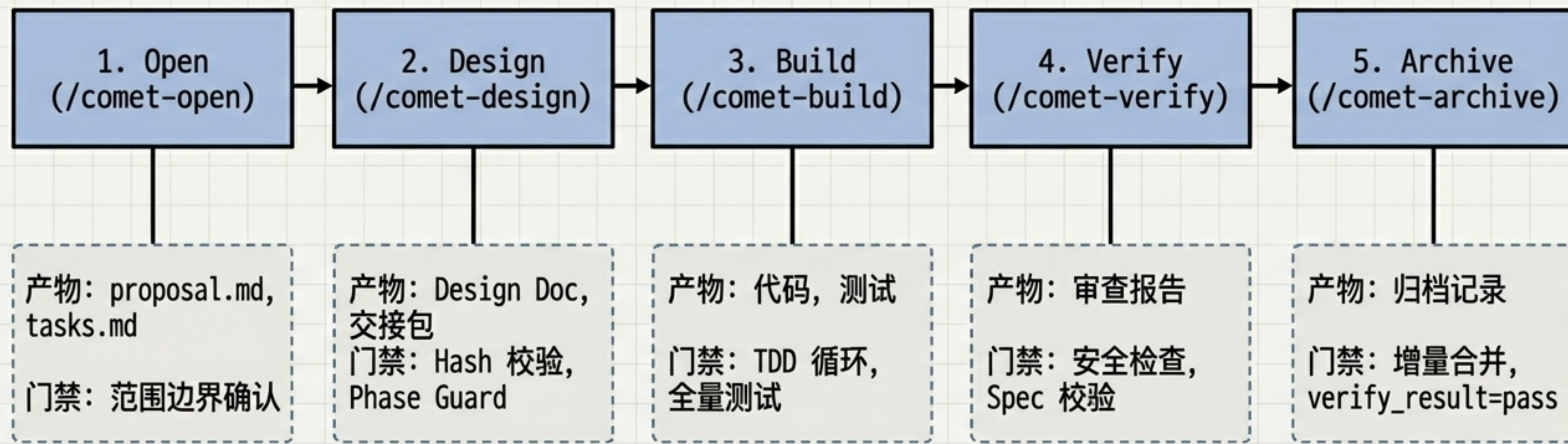
✅ 默认放行：编辑文件，读 .env，HTTP Get，提 PR。

❌ 默认拦截：curl | bash, 强制推 main, terraform destroy, 发送敏感数据。

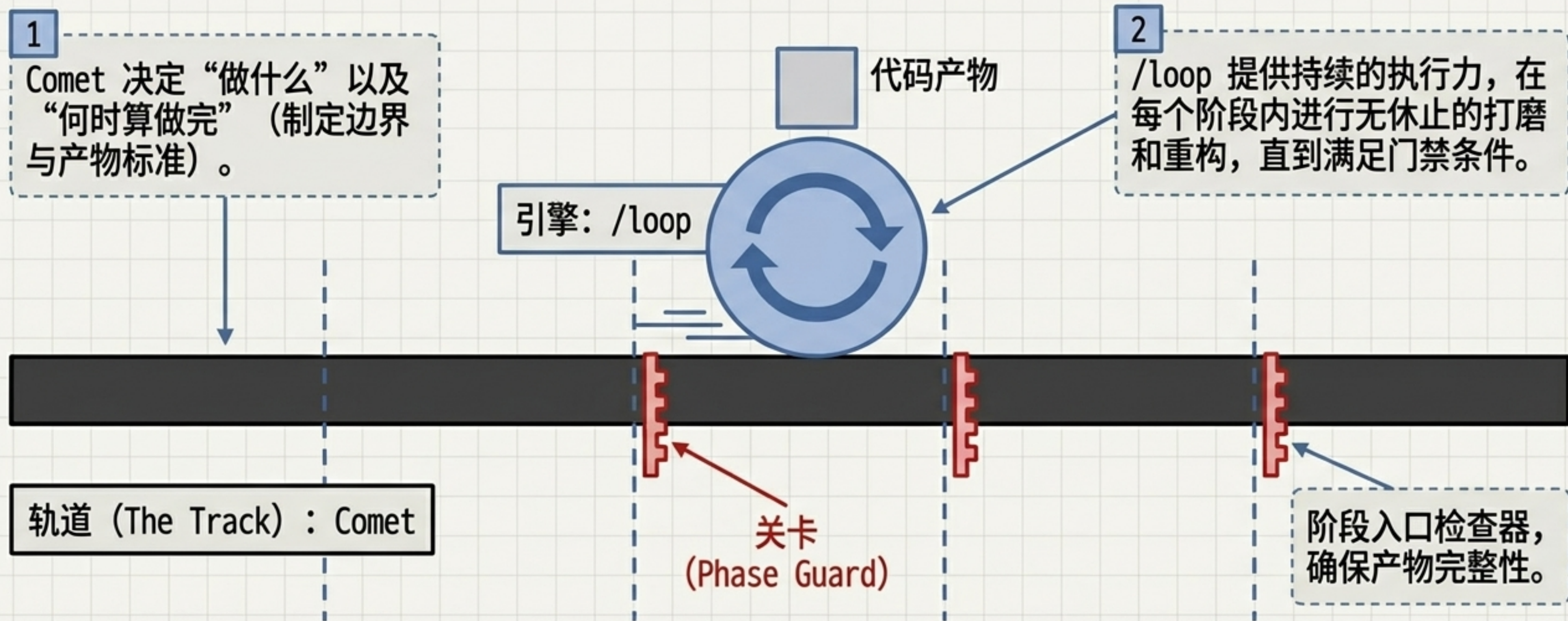
Requires ~/.claude/settings.json configuration.

引入 Comet: 5 站式 AI 软件工厂编排系统

以 `.comet.yaml` 为状态机，定义变更生命周期。



协作模型：轨道与引擎的完美契合



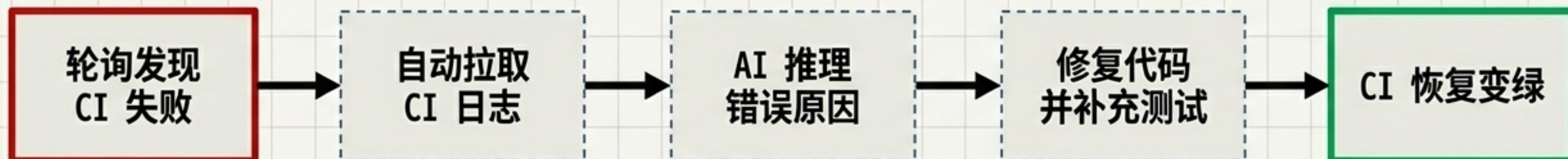
核心认知：/loop 不是盲目的死循环，而是被 Comet 赋予了确定性方向的自动装配线。

实战案例 1：夜间无人值守的 TDD 推进

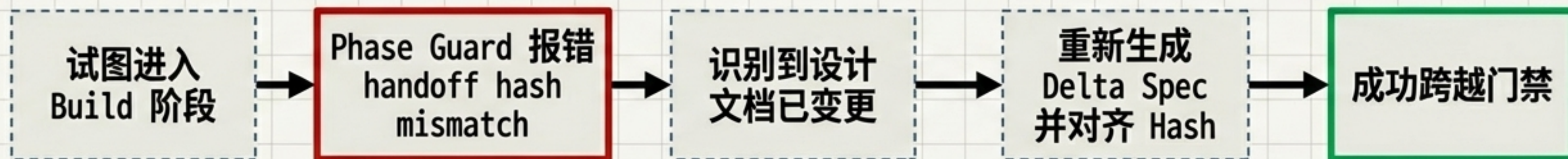


实战案例 2：应对异常环境的自动自愈（Auto-Healing）

Scenario A: CI 变红



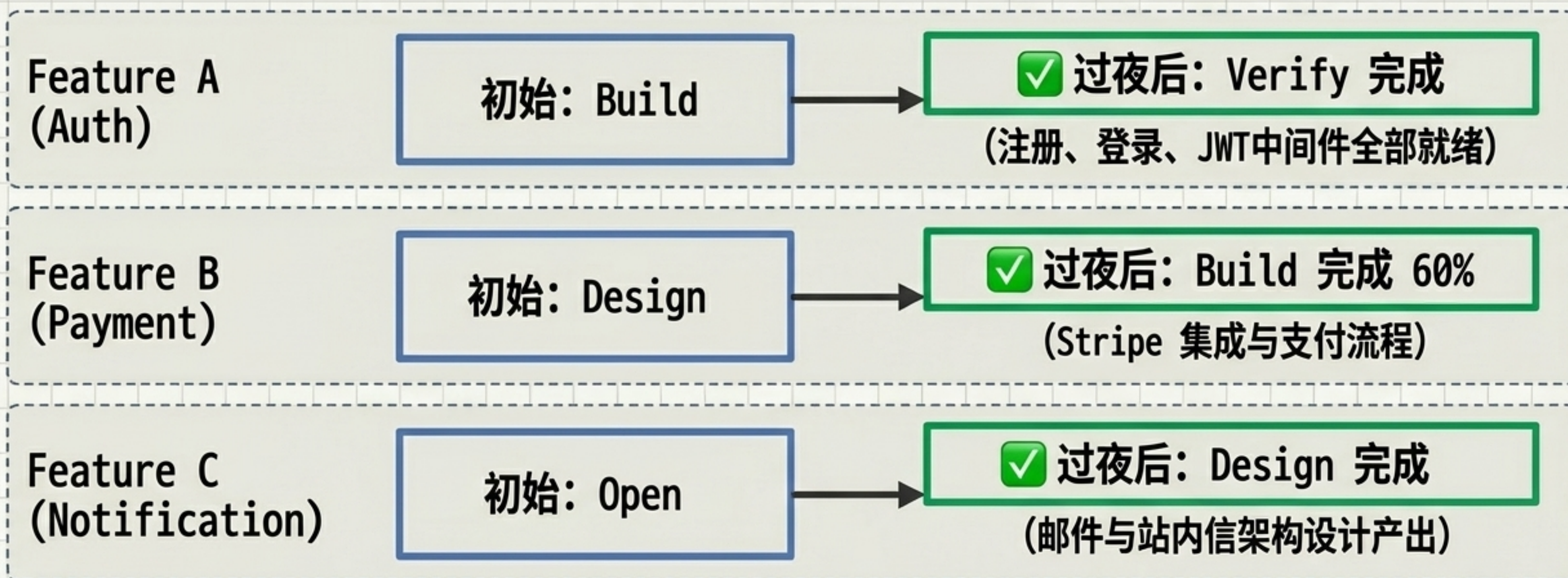
Scenario B: 质量门禁拦截



结论：突破传统 CI 定时的死板，具备基于上下文的动态纠错能力。

实战案例 3：多工作树并行开发（Parallel Multi-track）

三个独立 Feature 同时在不同的 git worktree 中由各自的 /loop 推进。



极大地拓宽了单兵开发者的并行带宽。

经济学视角：轮询间隔与成本估算

基于单次运行约 \$0.03 – \$0.10（取决于模型与上下文长度）。

应用场景	建议间隔	日运行次数	年化成本预估
日报摘要生成	24 小时	1 次	可忽略（极低）
代码质量扫描	1 – 6 小时	4 – 24 次	\$（低）
PR 维护与审查	10 – 30 分钟	48 – 144 次	\$（中低）
CI 持续监控	5 – 10 分钟	144 – 288 次	\$\$（中）
部署强轮询	1 – 5 分钟	288 – 1440 次	\$\$\$（较高，建议短时使用）

系统边界与工程绕过方案 (Workarounds)

进程绑定 (Process Lifecycle)

限制：会话关闭 loop 即死，最大 7 天过期。



方案：使用 tmux / screen 挂机保活，或结合 Routines 转为云端持久化。

上下文膨胀 (Context Bloat)

限制：每次迭代累积历史，Token 消耗急剧增加。



方案：定期执行 /compact 指令，或设计高度自包含的无状态 prompt。

调度抖动 (Scheduler Jitter)

限制：定期任务可能出现 0-30 分钟随机偏移。



方案：避免使用 :00，使用精确调度偏移（如 :03）绕过峰值拥堵。

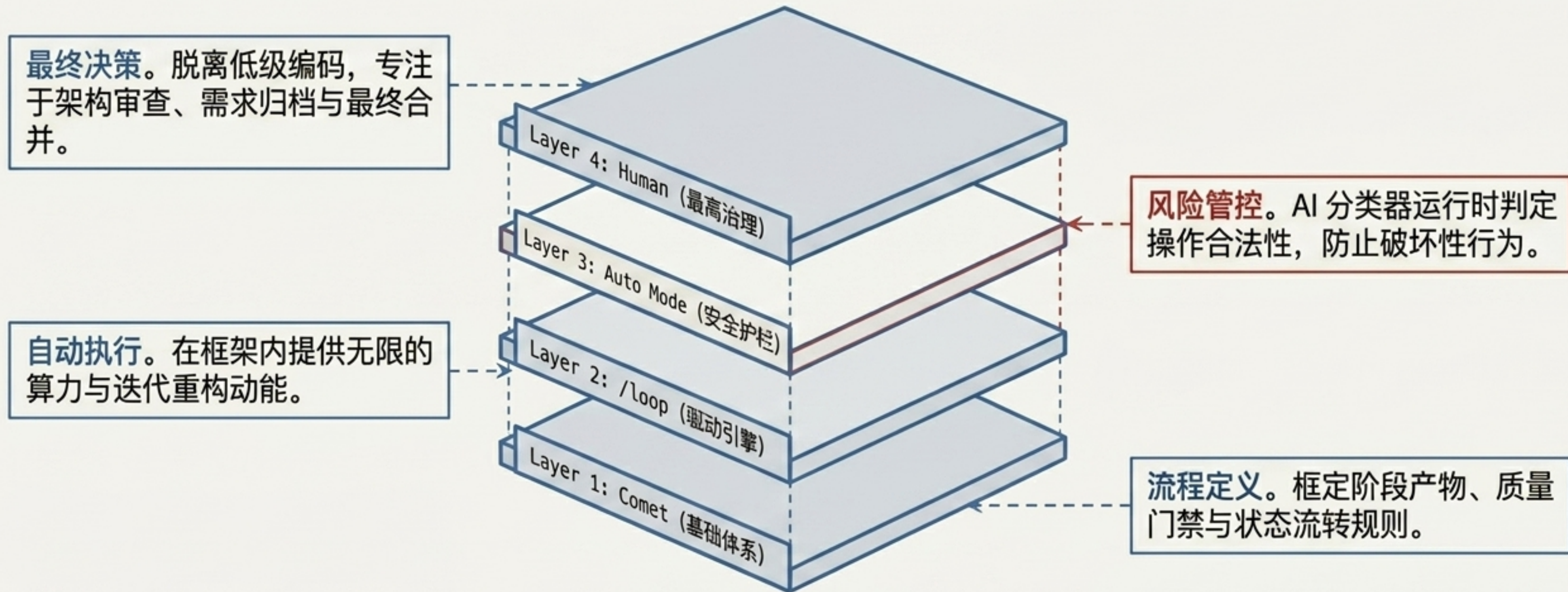
无原生告警 (Silent Execution)

限制：发生严重异常时只在终端默默输出。



方案：在 loop.md 注入 curl 命令，失败时调用飞书/钉钉 Webhook。

架构全景：无人值守 AI 研发栈的 4 层模型



> System Ready. Awaiting initial commit... _